

# WM as simulator

## 相关文献



<https://arxiv.org/abs/2511.09515>

### WMPO: World Model-based Policy Optimization for Vision-Language-Action Models

Vision-Language-Action (VLA) models have shown strong potential for general-...

用一个 **action-conditioned** 视频 **world model** 代替真实环境，让 **VLA policy** 在“想象出来的视频轨迹”里做 **on-policy GRPO**。

它不是传统 Dreamer/RSSM 那种 latent state world model，而是强调 **pixel-level imagined trajectory**，因为 VLA 本身吃图像，作者认为生成像素视频再喂给 VLA，比把 policy 放进一个新 latent space 更对齐。

### 整体框架

WMPO 有三块：

#### 1. VLA policy

用 OpenVLA-OFT 做 base policy。输入最近视觉观测和 language instruction，输出一个 action chunk。论文实现里把 action 每个维度离散成 256 bins，action chunk length 是 8。

#### 2. Generative world model

输入最近若干帧 + policy 给出的 action chunk，预测下一段视频帧。然后把预测帧继续作为下一步 conditioning，和 policy 交替运行，直到生成完整 imagined trajectory。

#### 3. Reward model

用 VideoMAE encoder + linear head 做二分类，判断一段 imagined trajectory 是否成功。训练数据来自真实 rollout：成功轨迹的 terminal clip 是正样本，成功轨迹中间片段和失败轨迹片段是负样本。推理时对 imagined video 滑窗打分，只要某个 clip 超过阈值，就判为成功。

### World Model 架构

WMPO 的 world model 基于 OpenSora 视频 diffusion backbone，但做了几个改动：

#### 1. OpenSora backbone + SDXL 2D VAE

它把 OpenSora 原来的 3D VAE 换成 SDXL 的 2D VAE。理由是 robot manipulation 里细粒度接触和小物体运动很重要，3D VAE 压缩太强可能造成 temporal distortion 或细节丢失。

## 2. Pixel-space rollout for VLA

diffusion 在 VAE latent 里预测，但用于 policy optimization 时会 decode 回 image pixel space。这个点很关键：WMPO 的卖点之一就是 imagined trajectory 和 VLA pretrained visual representation 对齐，而不是让 VLA 学一个新的 latent world state。

## 3. Noisy-frame conditioning

因为 rollout 是 autoregressive 的，后面会看到前面自己生成的帧。普通训练如果只看 clean real frames，推理时遇到 generated noisy frames 就会崩。WMPO 的做法是在训练时给 conditional frames 加一点 diffusion noise，论文写的是 50/1000 steps，目的是让 world model 对不完美历史帧更 robust。

## 4. Frame-level action control

为了让动作和视频帧对齐，它把 action signal 注入 diffusion transformer block 里。具体是扩展 AdaLN，用每一帧对应的 action 通过 MLP 生成 modulation 参数，对 LayerNorm / residual 做调制。简单说就是：不是只把 action 当一个全局 condition，而是逐帧控制视频生成。

## 5. Policy Behavior Alignment

World model 先在 OXE 上预训练，再用当前 policy 自己采集的 real rollout 继续 fine-tune。原因是 OXE 和 expert demos 大多是成功轨迹，缺少 failure distribution；但 RL 需要 world model 能模拟失败，否则 imagined rollout 对 policy update 没意义。

## Policy Optimization 流程

每次从真实环境采一个 initial state，然后：

1. 当前 policy 根据图像和语言输出 action chunk。
2. World model 根据历史帧和 action chunk 生成下一段帧。
3. 重复 policy → WM → policy → WM，得到完整 imagined trajectory。
4. Reward model 给每条 imagined trajectory 打 success/failure。
5. 对同一个初始状态采一组 trajectories，用 GRPO 做 group-relative advantage。
6. 如果一组全成功或全失败，就丢掉重新采，因为这种 group 没有相对优势信号。
7. 用 imagined trajectories 更新 policy，不再和真实环境交互。

所以 WMPO 的关键不是“训练一个世界模型”本身，而是把它接成一个 **closed-loop imagined RL environment**。

## 但它的问题也很明显

WMPO 仍然基本假设 world model 可以当 simulator 用。它虽然用了 noisy-frame conditioning，但没有系统处理“长程 rollout 越滚越偏”的问题，只是让 wm 可以在有一定噪声的条件帧下生成未来帧。



<https://arxiv.org/abs/2602.13977>

# WoVR: World Models as Reliable Simulators for Post-Training VLA Policies with RL

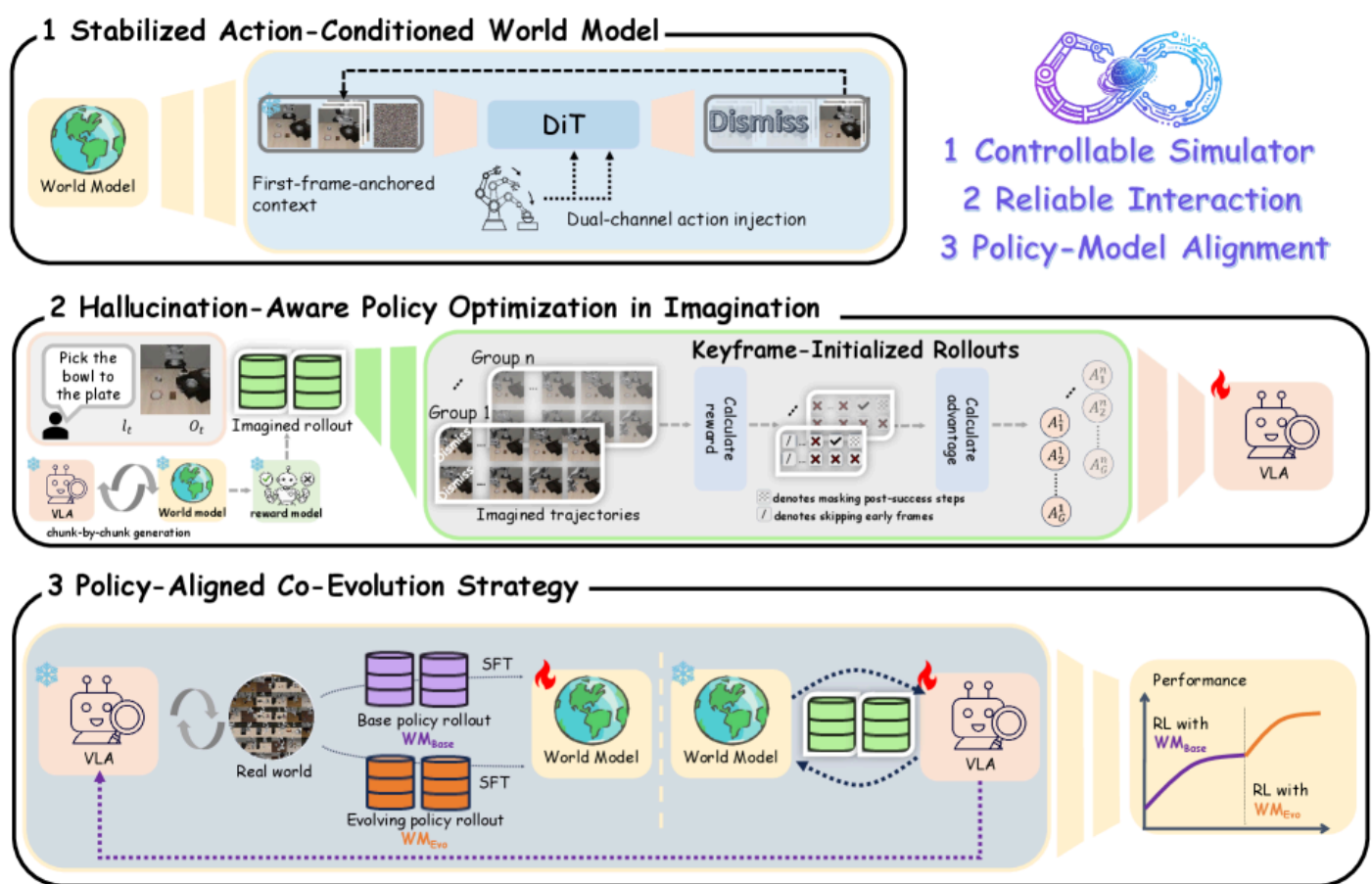
Reinforcement learning (RL) promises to unlock capabilities beyond imitation...

WoVR 可以看成是对 WMPO 的直接修正：WMPO 证明了可以用 video WM 做 imagined RL；WoVR 说问题在于这个 WM 不是可靠 simulator，所以要显式控制 hallucination 和长程误差累积。

## 一句话核心

WoVR 的核心是：把 action-conditioned video world model 当成一个“不完美 simulator”，然后从 world model 本身、rollout 起点、policy-WM 分布对齐三个层面限制误差。

## 整体架构



它有三层：

## 1. Stabilized Action-Conditioned World Model

这是 simulator 本体。

Backbone 用的是 Wan2.2-TI2V-5B video diffusion model，然后改成 action-conditioned world model。关键设计有三个：

- **dual-channel action injection**

action 通过两条路进 DiT block:

一条是 action embedding 和 diffusion timestep embedding 融合, 用 AdaLN-Zero 调制 denoising dynamics; 另一条是把原本 text cross-attention 的条件换成 action embedding。简单说: 一条负责局部调制, 一条负责全局条件。

- **first-frame anchoring**

每次 autoregressive rollout, 不只看最近几帧, 还一直保留 episode 的 initial frame 作为 reference。

输入 context 类似: `initial frame + recent memory frames`。

目的就是防止背景、物体布局、全局外观慢慢漂掉。

- **noisy context training**

训练时给非 reference 的 context latent 加 diffusion noise。

这和 WMPO 的 noisy-frame conditioning 类似, 都是为了减少 teacher forcing 训练时看 clean real frames、推理时看 self-generated imperfect frames 的 gap。

但注意: 这还不是 MaineCoon 那种 self-resampling, 它只是人工加 noise, 不是让模型先生成 corrupted history 再训练下一段。

## 2. Hallucination-Aware Policy Optimization

这是它怎么用 WM 做 RL。WoVR 不完全相信从 initial state 一路 rollout 到 episode 结束。它认为长程 rollout 前面的小错会把后面全带偏, 所以提出:

- **KIR, Keyframe-Initialized Rollouts**

不总是从 episode 初始帧开始 imagined rollout, 而是从 task-critical keyframes 附近开始, 尤其是当前 policy 的 failure states 附近。

这样 world model 只需要预测较短的一段关键动态, 减少 effective error depth。

- **masked GRPO**

如果 imagined rollout 已经预测成功, 后面的 post-success steps 不再用于优化, 避免 policy 被“成功以后继续 hallucinate 的画面”误导。

同时按 valid length 做 normalization, 让短的、关键的 rollout 更有权重。

## 3. PACE, Policy-Aligned Co-Evolution

这是处理 policy distribution shift。因为 policy 在 WM 里优化后, 动作分布会变, 原来的 WM 可能没见过这些 action/state。PACE 的做法是低频 realignment:

- 先用 base policy 的真实 rollout 训练初始 WM;
- policy 在 WM 里优化一轮;
- 再用 updated policy 采少量真实 rollout;
- 用这些新数据 refine WM;
- 然后继续在 refined WM 里做 policy optimization。

## 4. 它不是高频在线更新 dynamics model, 而是低频校准 WM 和当前 policy 的分布。

## 和 WMPO 的关键区别

WMPO 的主线是：用 pixel video WM 替代真实环境，让 VLA 做 imagined GRPO。

WoVR 的主线是：这个替代品不可靠，所以要限制它的使用方式。

具体来说：

- WMPO 有 noisy-frame conditioning，但仍然偏“直接把 WM 当 simulator 用”。
- WoVR 加了 first-frame anchoring，让 WM rollout 本身更稳。
- WoVR 用 KIR，把长 episode 切成靠近关键状态的短 rollout。
- WoVR 用 masked GRPO，避免 post-success hallucination 污染 reward。
- WoVR 用 PACE，周期性把 WM 拉回当前 policy 的分布。



<https://arxiv.org/abs/2605.07288>

Sword: Style-Robust World Models as Simulators via  
Dynamic Latent Bootstrapping for VLA Policy Post-

The integration of Vision-Language-Action (VLA) models with World Models has...

它不是改 RL，而是训练一个更鲁棒的 **action-conditioned video world model**，让它更适合作为 **VLA policy post-training 的 simulator**。

## 核心问题

Sword 认为 WoVR 这类 WM simulator 还有两个主要问题：

### 1. style / OOD generalization 差

LIBERO 里背景、光照、桌面颜色、机械臂颜色稍微变一下，world model 可能很快 blur、过曝、物体结构崩掉。作者认为这是因为 WM 记住了表面纹理，而不是学到了稳定的物理/几何 dynamics。

### 2. teacher forcing 导致 exposure bias

训练时 context 是真实历史帧；推理时 autoregressive rollout 只能吃自己生成的帧。小误差会被反复喂回模型，长程 rollout 就会越来越偏。

## 整体架构

Sword 的 world model 还是一个 action-conditioned video diffusion simulator：

代码块

```
1  initial frame anchor + recent context frames + action sequence
2      -> video diffusion world model
3      -> next future frames
4      -> autoregressive rollout
```

它的 backbone 是 **Wan 2.2 TI2V**，DiT 架构。action sequence 被转成 text-like condition，然后通过两条路注入 Transformer block：

- **AdaLN branch**: 用 action 调制每层 denoising dynamics;
- **Cross-Attention branch**: 让 action 作为条件参与 attention。

推理时每一步:

- 输入 **4 frames context**;
- 预测 **next 8 frames**;
- 同时把 episode 的 **first frame** 拼进去作为 global anchor, 保持长程外观和场景一致性。

这点和 WoVR 的 first-frame anchoring 是一致的。

### 核心方法 1: Structure-Guided Style Augmentation

这是解决 OOD style 的。

它不是简单做颜色 jitter, 而是用 Cosmos-Transfer 2.5 做结构保持的 style augmentation。变化包括:

- brightness
- saturation
- robot arm color
- tabletop / background appearance
- illumination

为了防止 style transfer 改坏物体几何和任务语义, 它额外加入结构约束:

- depth map, 用 DepthAnything;
- semantic mask, 用 GroundingDINO / SAM2;
- task prior, 描述交互过程和任务语义。

目的就是让训练数据在视觉风格上更多样, 但物体位置、几何结构、robot-object interaction 不变。这样 WM 被迫学 dynamics, 而不是死记 LIBERO 原始纹理。

### 核心方法 2: Dynamic Latent Bootstrapping, DLB

这是解决长程 rollout error accumulation 的, 也是最关键的部分。

普通 teacher forcing:

代码块

```
1 GT history latent -> predict future latent
```

但 inference 是:

代码块



```
1 model-predicted history latent -> predict future latent
```

Sword 的 DLB 做法是维护一个 **dynamic latent cache**，里面存模型自己预测出来的 VAE latents。训练早期主要用真实 history latents，同时不断往 cache 里写入 predicted latents；训练后期逐渐提高概率，用 cached predicted latents 替换 GT history latents 作为 context。

也就是逐步变成：

代码块

```
1 mixed / predicted history latent -> predict GT future latent
```

这个设计的好处是：

- 训练分布更接近 inference 分布；
- 模型学会从自己产生的小错误里恢复；
- 在 VAE latent space 存 cache，内存比 pixel-space rollout 小很多；
- 不需要突然从 teacher forcing 切到 full autoregressive training，训练更平滑。

**这篇的核心贡献**

最重要的是两点：

### 1. 让 world model 对视觉风格扰动更稳

通过 structure-guided style augmentation，减少对 LIBERO 固定视觉纹理的过拟合。

### 2. 让 world model 对自己生成的 history 更稳

通过 DLB，把 self-generated latent context 逐步引入训练，缓解 teacher forcing 和 closed-loop inference 的 mismatch。



<https://arxiv.org/abs/2603.20607>

**Towards Practical World Model-based Reinforcement Learning for Vision-Language-Action Models**

Vision-Language-Action (VLA) models show strong generalization for robotic...

这篇是 **VLA-MBPO: Towards Practical World Model-based Reinforcement Learning for VLA Models**。它和 Sword 不一样：**Sword 主要改 world model 训练；VLA-MBPO 是一个完整的 model-based RL framework**，里面同时改了 world model 形式和 rollout 使用方式。

**一句话核心**

VLA-MBPO 的核心是：

用一个 **unified multimodal world model** 同时预测 **future observation** 和 **reward**，再用 **chunk-level branched rollout** 限制误差累积，让 VLA 可以在 learned WM 里做 PPO/Flow-Noise 优化。

## 它解决的问题

论文认为 VLA 上直接做 world-model RL 有三个实际问题：

### 1. pixel-level world modeling 难

VLA 吃的是原始图像，所以 WM 不能只预测低维 latent state。

### 2. multi-view consistency 难

很多机器人策略同时用 head/top camera 和 wrist camera。两个视角如果各自独立生成，会出现跨视角不一致，影响控制。

### 3. sparse reward + long rollout 会放大 model error

小的视觉预测误差可能导致 reward 判断完全相反，所以 full-horizon rollout 很危险。

## World Model 架构：UMM-World

这篇没有沿用 WMPO/WoVR/Sword 那种纯 video diffusion simulator，而是用 **Unified Multimodal Model, UMM** 做 world model，具体实现用的是 **Bagel**。

输入包括：

代码块

```
1  language instruction
2  + multi-view current observations
3  + discretized action chunk
```

输出包括：

代码块

```
1  next observation
2  + chunk-level reward / success signal
```

action 的处理方式比较简单：把连续 action 每个维度离散成整数 token，例如 0 到 256，然后当成 text-like token 塞进 UMM vocabulary。这样不用大改模型结构。

它的一个卖点是：**world model 和 reward model 合在一个 UMM 里**。不像 WMPO/WoVR 是一个视频 WM 加一个单独 reward model；VLA-MBPO 让同一个模型既预测视觉 dynamics，也预测 reward。

## 关键设计 1：Interleaved View Decoding

这是为了解决 multi-view consistency。

如果 head view 和 wrist view 分开生成，两个视角可能局部都合理，但对应不上同一个物理状态。VLA-MBPO 用 interleaved decoding，让不同 view 的 token 在同一个生成过程中互相约束。

直观地说：



- 1 不要并行独立生成 head view / wrist view
- 2 而是让两个视角交错解码、共享上下文

这样 head view 提供全局场景，wrist view 提供接触细节，两者保持一致。

### 关键设计 2: Chunk-Level World Model

VLA policy 本来就是 action chunk policy，一次输出一段 action。VLA-MBPO 让 world model 也在 chunk level 工作：

代码块

```
1 state_t + action_chunk -> state_{t+k}, reward_chunk
```

而不是 step-by-step 预测每个中间帧。

好处是 rollout 更短、更快，也减少误差累计。论文里也强调 UMM 可以通过 frame-skipping 直接预测后面的 observation，不需要像 video model 那样生成所有 intermediate frames。

### 关键设计 3: Chunk-Level Branched Rollout

这是这篇对长程累积误差最重要的处理。

它不从 episode 初始状态一路 rollout 到终点，而是：

代码块

- 1 从 offline buffer 里的任意真实 observation 开始
- 2 只 rollout 很短的几个 action chunks
- 3 用这些短 synthetic rollouts 做 policy update

这个和你说的 “trust radius” 非常接近：**不要相信 WM 的 full-horizon simulation，只相信短 chunk 内的预测。**

它的 ablation 也支持这个点：在 LIBERO-Long 上，branched rollout 长度为 2 chunks 最好，full-horizon rollout 明显更差。

### Policy Optimization

整体流程是三阶段：

1. 用 VLA 收集数据，放进 replay buffer。
2. 用这些数据 fine-tune UMM-World。
3. 在 world model 里采样短 branched rollouts，用 Flow-Noise / PPO 风格的方法更新 VLA policy。

它还给 VLA 加了一个 MLP value head 来估计 advantage。

## Reliable World Simulator

# Motivation

VLA policy post-training 需要大量交互，但真实机器人交互昂贵、缓慢，并且存在安全风险。因此，近期工作开始使用 learned world model 作为 simulator，在 imagined rollouts 中优化 VLA policy，例如 WMPO、WoVR，以及后续的 Sword。这条线的核心思路是：用 action-conditioned video world model 预测 policy action chunk 导致的未来观测，从而减少真实环境交互成本。

然而，当前 video WM-as-simulator 仍然不是一个完整可靠的 RL simulator。对 RL 来说，一个 simulator 不只是为了回答：如果执行这个 action，未来会长什么样？

还必须回答：

- 这个未来是否物理一致？
- 这个未来对任务是否有价值？

换句话说，一个对 policy optimization 有用的 simulator 至少需要同时提供：

- transition:  $obs_t + action\_chunk \rightarrow future\_obs / future\_state$
- reward: imagined transition  $\rightarrow$  task reward / value

现有 video WM 方法主要强化了第一部分。WMPO 证明了 action-conditioned video WM 可以用于 imagined policy optimization；WoVR 进一步通过 KIR、first-frame anchoring 和 PACE 降低长程 rollout 风险；Sword 则通过 style augmentation 和 Dynamic Latent Bootstrapping 改善视觉泛化和 self-generated context 下的稳定性。这些工作逐步提升了 video rollout 的可用性，但它们的 simulator 仍然主要围绕 visual transition 展开：给定视觉历史和 action chunk，生成未来视频，再依赖外部 reward classifier 或下游判断来评估 imagined rollout。

这带来两个关键限制。

第一，**transition reliability 仍然不够完整**。Video rollout 视觉上 plausible，并不意味着它物理上一致、动作上可信。对于机器人任务，单视角视频容易受遮挡和视觉歧义影响；而 action consequence 又强依赖当前机器人状态。只预测未来 RGB 会让 world model 缺少 embodiment anchor，可能生成看起来合理但与末端位姿、接触状态或真实动作结果不一致的 rollout。因此，一个更可靠的 simulator 应该不仅预测 future video，还应该显式利用 current proprio / robot state，并预测 future proprio / state，使 imagined transition 同时满足视觉一致性和机器人状态一致性。

## Action image

第二，**reward/value interface 仍然不够内生**。许多 WM-as-simulator 方法将 reward 放在 world model 外部，例如使用 ResNet、VideoMAE 或 VLM classifier 判断 generated observation 是否成功。这种做法可以工作，但 reward model 和 generated dynamics 是分开训练、分开泛化的；当 policy 在 imagined rollouts 中优化时，reward 需要在 generated observation distribution 上可靠工作，否则容易出现 dynamics-reward inconsistency：world model 生成的未来和 reward model 给出的任务价值并不真正匹配。对 RL 来说，这会直接影响 policy update 的质量。

近期 ViVa 和 Cosmos Policy 提供了一个重要启发：video/generative backbone 不只适合生成未来观测，也可以承载 value/progress 信息。ViVa 表明，任务价值依赖时间动态和未来预期，因此

generative video representation 可以为 value estimation 提供更强时序语义；Cosmos Policy 则说明 DiT/video-style 模型具备联合建模 future state 和 value 的潜力。但它们并不直接等价于我们需要的 WM simulator：ViVa 不是 action-conditioned simulator，Cosmos Policy 更偏 policy-centric，会建模或生成 action。对于 WM-as-simulator，我们真正需要的是：given a policy action chunk, predict its future consequences and estimate its task value.

因此，我们的目标是训练一个 **Reliable Action-Conditioned Embodied Video-Value World Simulator**。它不是替代 policy，也不是提出新的 RL 算法，而是为 VLA policy post-training 提供一个更完整的 imagined environment：输入 multi-view observations, current proprio / robot state, action chunk, task instruction；输出 future multi-view observations, future proprio / robot state, action-conditioned reward / value

这个 simulator 的核心目标是：在短期 imagined rollout 中，稳定预测 action-conditioned future dynamics，并同时给出与该 imagined transition 一致的 task value。相比只生成未来视频的 video WM，它更接近 RL 所需的 simulator interface；相比外挂 reward classifier，它将 dynamics 和 value 放到同一个 action-conditioned world model 中，使 reward/value 直接服务于 imagined transition 的评估。

一句话总结：

现有 video WM-as-simulator 主要回答“未来长什么样”；而一个对 VLA policy post-training 真正可靠的 simulator，还必须回答“这个未来是否物理一致、是否对任务有价值”。因此，我们希望训练一个 action-conditioned embodied video-value world simulator，将 multi-view dynamics、robot proprioception 和 reward/value 统一建模，为 policy 提供更可靠的 imagined environment。

Memory wam

## Method

### 1. World Model Backbone and Simulator Adaptation

我们首先选择一个合适的 video world model backbone。已有 WoVR/Sword 这类工作证明，Wan-family text/image-to-video backbone 可以通过 action conditioning 被改造成 action-conditioned robot world model。然而，这类 generic video backbone 的预训练分布主要来自普通视频，缺少机器人交互中的接触、抓取、遮挡、物体运动和 embodiment dynamics。

因此，我们选择 **WoW-Wan** 作为 world model initialization。WoW-Wan 保留了 Wan-style video DiT 的架构形式，同时在大规模机器人交互轨迹上进行过预训练，因此相比 generic Wan 更可能具备机器人操作和物理交互的先验。这里我们不把 WoW-Wan 当作一个已经完成的 RL simulator，而是把它作为一个 robot-interaction-pretrained video dynamics prior。

在此基础上，我们将 WoW-Wan 从普通 video generation backbone 改造成 VLA post-training 所需的 action-conditioned embodied simulator。原始 video model 主要回答：

```
1 given context / prompt, what future video is plausible?
```

而我们需要 simulator 应该回答：

代码块

```
1 given current observation and a policy-proposed action chunk,  
2 what future transition will this action lead to?
```

因此，我们将 simulator 接口设定为：

代码块

```
1 S_theta(  
2     multi-view observations,  
3     current proprio / robot state,  
4     action chunk,  
5     task instruction  
6 )  
7 ->  
8     future multi-view observations,  
9     future proprio / robot state
```

其中，multi-view observations 用来减少单视角遮挡和视觉歧义；current proprio / robot state 提供 embodiment anchor，因为同一个 action chunk 在不同 robot state 下会导致不同后果；future proprio / robot state 则让 simulator 不只是生成视觉上 plausible 的视频，而是预测与机器人状态一致的 future transition。

为了进一步提高 action-conditioned transition 的可靠性，我们参考组内前置工作 **IOI** 的 kinematic grounding 思路。IOI 的核心观点是：机器人自身运动很大一部分由 deterministic kinematics 决定，而物体接触和环境响应才是更随机、更难建模的 physical dynamics。因此，world model 不应该完全依赖一个低维 action embedding 去隐式学习机器人运动学；当 action 是 joint command、EEF pose 或可以通过 IK / FK 解释的控制量时，我们可以利用 current proprio / robot state、action chunk 和 robot URDF 显式计算 future kinematic trajectory，再将其作为结构化条件注入 video DiT。

这样，WoW-Wan 的 simulator adaptation 不只是 naive action conditioning，而是：

代码块

```
1 robot-pretrained video prior  
2 + explicit kinematic grounding  
3 -> action-faithful embodied transition model
```

这可以减少模型在 robot morphology、end-effector motion 和 future proprio 上的 hallucination，使 imagined transition 更符合 action consequence。

## 2. Action-Conditioned Value Interface

在获得 action-conditioned transition model 之后，我们进一步为 world model 加入 value interface。现有 WoVR/Sword 这类 video WM-as-simulator 方法通常将 reward 放在 world model 外部，例如使用独立的 success classifier 判断 generated observation 是否完成任务。虽然这种方式可以提供基本 reward signal，但 reward model 和 imagined dynamics 是分开训练、分开泛化的；当 policy 在 imagined rollouts 上优化时，外部 reward 可能无法稳定评估 generated observation distribution，从而产生 dynamics-reward inconsistency。

受 Cosmos Policy 和组内前置工作 **MV-WAM** 启发，我们将 value 作为 world model 需要预测的一个模态，而不是外挂在 simulator 后面的独立判别器。Cosmos Policy 从 high-level 上说明 video diffusion backbone 可以通过 latent-frame injection 同时承载多视角图像、robot proprioception、action chunk、future observations 和 expected return / value。MV-WAM 则进一步提供了更直接的组内依据：它将 value token 放入 unified video-action model 中，并用 Monte Carlo returns 监督 progress-value estimation，用于检测执行偏离和 value-guided correction。

不同的是，Cosmos Policy 和 MV-WAM 都更偏 policy-facing / WAM setting，会生成或辅助生成 action；而我们的目标是 simulator-centric 的：action 由外部 VLA policy 提出，world model 只负责评估这个 action chunk 会导致什么未来，以及这个未来对任务是否有价值。

因此，我们将 simulator 输出进一步扩展为：

代码块

```
1  S_theta(  
2    multi-view observations,  
3    current proprio / robot state,  
4    action chunk,  
5    task instruction  
6  )  
7  ->  
8    future multi-view observations,  
9    future proprio / robot state,  
10   action-conditioned value
```

这里的 value 可以理解为对 policy-proposed action chunk 的评估，也可以理解为基于 predicted future state 的 task progress / expected return。关键点是，我们不把 value 设计成一个只看最终图像的 Yes/No classifier，而是让它依赖于同一个 world model 中预测出的 future visual and proprioceptive transition。这样，value prediction 与 imagined dynamics 共享表示和训练分布，使 simulator 不仅回答“执行这个 action 后未来长什么样”，也回答“这个未来是否朝任务成功推进”。



Value supervision 可以参考 MV-WAM 的做法，来自真实或 policy rollout 数据中的 trajectory outcomes / Monte Carlo returns。也就是说，我们不需要人工标注每个子任务 reward，而是利用真实 rollout 的成功/失败结果或 return-to-go 来监督 value，使模型学习哪些 state-action chunks 更可能推进任务完成。这里的 value 不是完整 episode 末端的单一 success classifier，而是面向 imagined transition 的 progress / expected return signal。

这个设计将 learned world model 从 video-only transition predictor 扩展为面向 RL post-training 的 simulator：给定外部 VLA policy 提出的 action chunk，它同时预测 future transition 和 action-conditioned task value，从而为 imagined rollout 中的 policy update 提供更一致的 reward/value signal。

### 3. Closed-Loop Reliability Training

在得到 action-conditioned embodied video-value simulator 之后，我们还需要解决 closed-loop rollout 中的误差累积问题。标准 video world model 在训练时通常条件于真实历史帧，但在 imagined rollout 中，模型会反复条件于自己生成的 future observations。这样会产生 train-rollout context gap：一旦前一个 chunk 的生成结果出现小偏差，这个偏差就会被作为下一个 chunk 的输入继续放大，最终导致长程 rollout 漂移，生成视觉上 plausible 但物理上不可信、value 判断也不可靠的 imagined trajectory。

我们的目标不是把 learned simulator 当作可以无限 rollout 的真实环境替代品，而是把它作为一个局部可信的 imagined environment。因此，我们显式限制 simulator 的 trust horizon，采用 short-chunk rollout，只在少量 action chunks 的范围内使用 imagined transitions 来辅助 policy update，而不是一次性 rollout 完整 episode。这一原则与 WoVR 等工作的使用方式一致：video WM 的优势在于提供局部 counterfactual dynamics，而不是长期完全替代真实环境。

进一步地，我们参考 Sword 的 Dynamic Latent Bootstrapping 以及 self-resampling 类方法，在训练过程中逐步引入 self-generated context。也就是说，world model 不只在 ground-truth history 上学习预测下一段未来，也会在由自身生成的 visual/proprio context 上继续预测后续 transition。训练分布从真实 context 逐渐过渡到 mixed GT/generated context，使模型学会在带有小错误、模糊或轻微漂移的历史条件下恢复到合理的未来，而不是只会在干净真实帧上做 teacher-forced prediction。

这里我们的重点是将 closed-loop reliability 从 video-only consistency 扩展到 embodied video-value simulator。因为我们的 simulator 同时预测 multi-view future observations、future proprio / robot state 和 action-conditioned value，所以 reliability 不只意味着视频帧清晰或连续，还要求 imagined transition 在多个层面保持一致：不同视角之间应描述同一个物理过程；future proprio 应与当前 robot state 和 action chunk 相匹配；value prediction 应与 predicted future transition 的任务进展一致。

IOI 和 MV-WAM 分别为这两个一致性目标提供依据。IOI-style kinematic grounding 可以作为 transition reliability 的结构约束，减少 self-generated rollout 中 action consequence 漂移；MV-WAM-style progress-value token 可以作为 value reliability 的监督来源和偏离检测信号，帮助识别 value 明显下降、future proprio 不一致或 imagined transition 不可信的样本。

通过 short-horizon trust、self-generated context training 和 joint visual / embodiment / value consistency，我们希望 learned simulator 在 online RL 过程中更可靠地服务于 policy post-training。它不被假设为完美环境，而是作为一个局部可信的 simulator：给定 policy 提出的 action chunk，稳定预测短程未来，并为这些 imagined transitions 提供一致的 value signal。

Ctrl world 作为 backbone，然后加 memory 然后加 value，vlm 用来 value 的训练日记。